

---

# Documentação de Projeto

para o sistema

<**BeautyBook**>

Versão 1.0

Projeto de sistema elaborado pelo(s) aluno(s) Alice Shikida Salomão  
como parte da disciplina **Projeto de Software**.

<**25/05/2026**>

## Tabela de Conteúdo

|                                                              |          |
|--------------------------------------------------------------|----------|
| <b>1. Introdução</b>                                         | <b>1</b> |
| <b>2. Modelos de Usuário e Requisitos</b>                    | <b>1</b> |
| 2.1 Descrição de Atores                                      | 1        |
| 2.2 Modelo de Casos de Uso e Histórias de Usuários           | 1        |
| 2.3 Diagrama de Sequência do Sistema e Contrato de Operações | 1        |
| <b>3. Modelos de Projeto</b>                                 | <b>1</b> |
| 3.1 Arquitetura                                              | 1        |
| 3.2 Diagrama de Componentes e Implantação.                   | 2        |
| 3.3 Diagrama de Classes                                      | 2        |
| 3.4 Diagramas de Sequência                                   | 2        |
| 3.5 Diagramas de Comunicação                                 | 2        |
| 3.6 Diagramas de Estados                                     | 2        |
| <b>4. Modelos de Dados</b>                                   | <b>2</b> |

## Histórico de Revisões

| Nome                     | Data           | Razões para Mudança | Versão |
|--------------------------|----------------|---------------------|--------|
| Alice Shikida<br>Salomão | 25/05/20<br>26 | Criação inicial     | 1.0    |
|                          |                |                     |        |

# 1. Introdução

Este documento agrega a elaboração e revisão de modelos de domínio e modelos de projeto para o sistema **BeautyBook**, uma aplicação web para gerenciamento de agendamentos em salões de beleza. O sistema permite que clientes façam reservas de serviços online e que funcionários e o administrador gerenciem a agenda, os serviços oferecidos e os relatórios de atendimento.

## 2. Modelos de Usuário e Requisitos

### 2.1 Descrição de Atores

Cliente: Usuário externo que realiza agendamentos de serviços pelo sistema.

Funcionário: Profissional do salão que visualiza e confirma os agendamentos atribuídos a ele.

Administrador: Responsável por gerenciar serviços, funcionários, horários e relatórios.

### 2.2 Modelo de Casos de Uso

#### Histórias de Usuário: Clientes

**US-01 – Cadastrar cliente** *Como cliente, quero me cadastrar na plataforma BeautyBook para poder acessar os serviços e realizar agendamentos.*

---

**US-02 – Realizar agendamento** *Como cliente, quero agendar um horário com um funcionário para garantir meu atendimento na data e hora desejadas.*

---

**US-03 – Cancelar agendamento** *Como cliente, quero cancelar um agendamento previamente realizado para liberar o horário caso não possa comparecer.*

---

**US-04 – Visualizar agenda** *Como funcionário, quero visualizar minha agenda de atendimentos para me organizar e me preparar para os serviços do dia.*

---

**US-05 – Gerenciar serviços** *Como administrador, quero cadastrar, editar e remover serviços oferecidos pelo estabelecimento para manter o catálogo sempre atualizado.*

---

**US-06 – Visualizar relatórios** *Como administrador, quero visualizar relatórios de atendimentos e faturamento para acompanhar o desempenho do negócio.*

---

**US-07 – Notificar cliente por e-mail** *Como sistema, quero enviar uma notificação por e-mail ao cliente sempre que um agendamento for cancelado ou reagendado para mantê-lo informado sobre alterações no seu horário.*

---

**US-08 – Aplicar cupom de desconto** *Como cliente, quero inserir um cupom de desconto durante o agendamento para obter redução no valor do serviço.*

---

**US-09 – Exportar relatório em PDF** *Como administrador, quero exportar os relatórios visualizados em formato PDF para armazená-los ou compartilhá-los facilmente.*

---

**US-10 – Avaliar atendimento** *Como cliente, quero avaliar o atendimento recebido após a conclusão do serviço para registrar minha satisfação e contribuir com a qualidade do estabelecimento.*

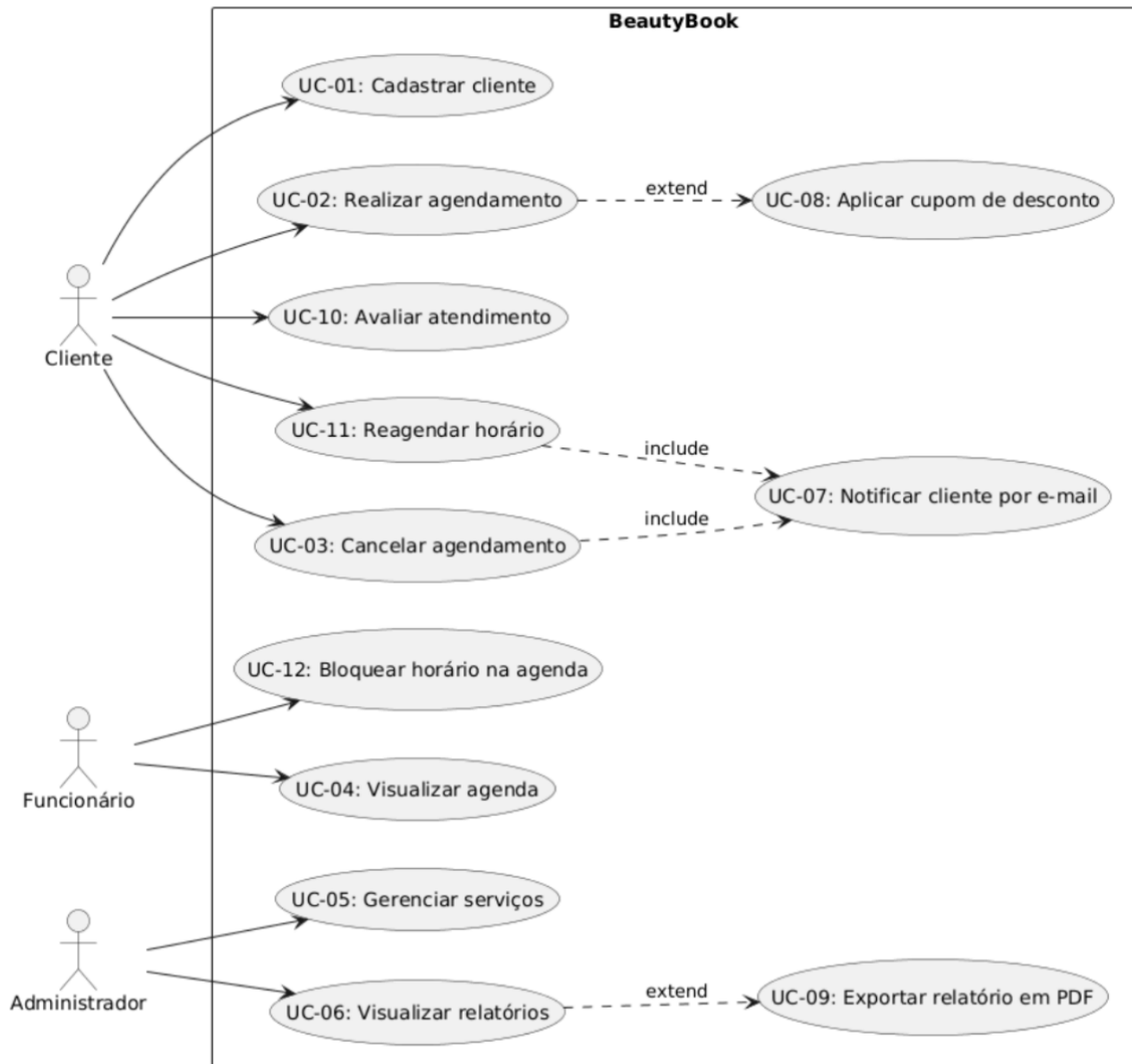
---

**US-11 – Reagendar horário** *Como cliente, quero reagendar um horário já marcado para alterar a data ou hora do meu atendimento sem precisar cancelar e remarcar manualmente.*

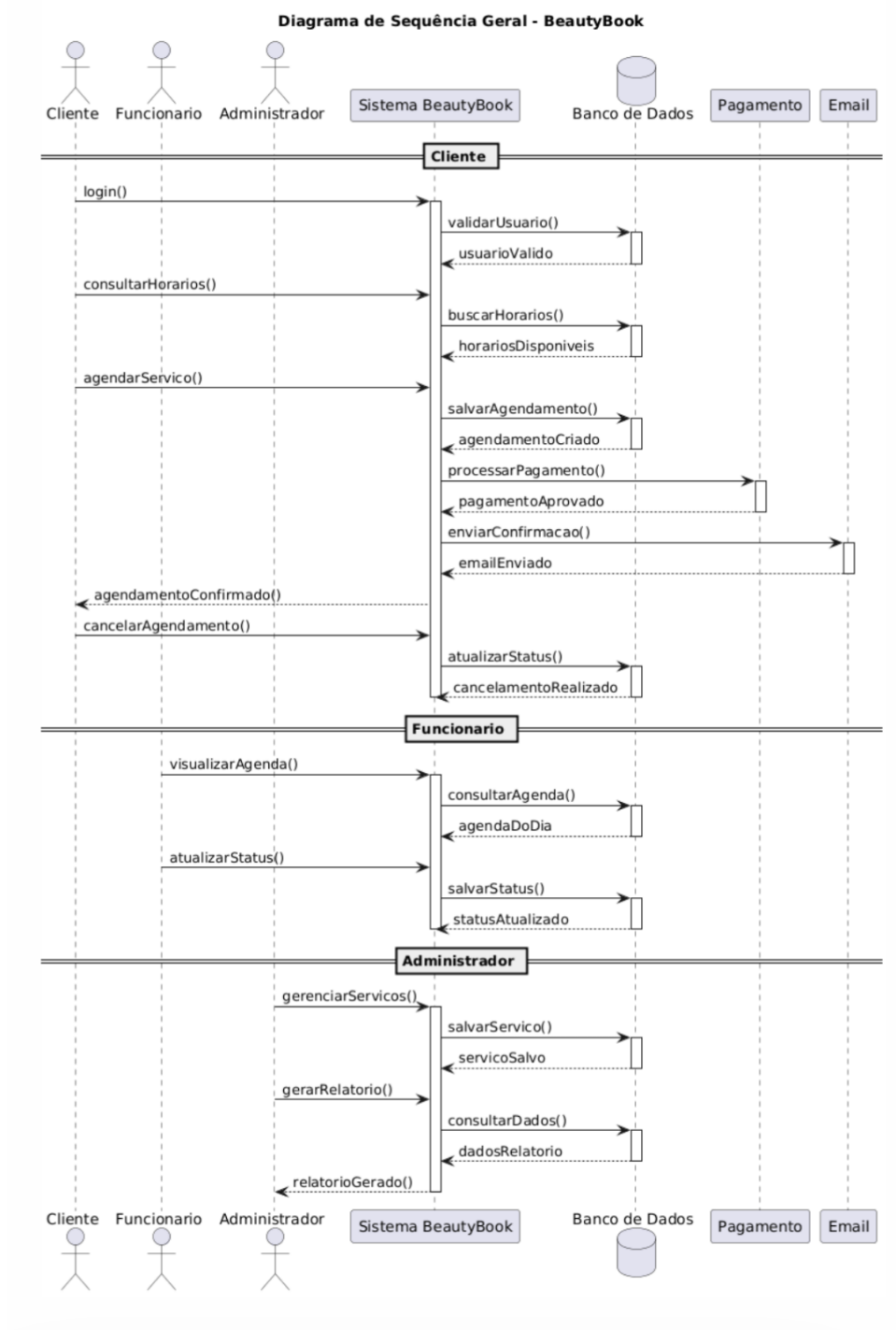
---

**US-12 – Bloquear horário na agenda** *Como funcionário, quero bloquear um horário na minha agenda para indicar minha indisponibilidade em determinado período e evitar agendamentos indevidos.*

Diagrama de Caso de Uso:

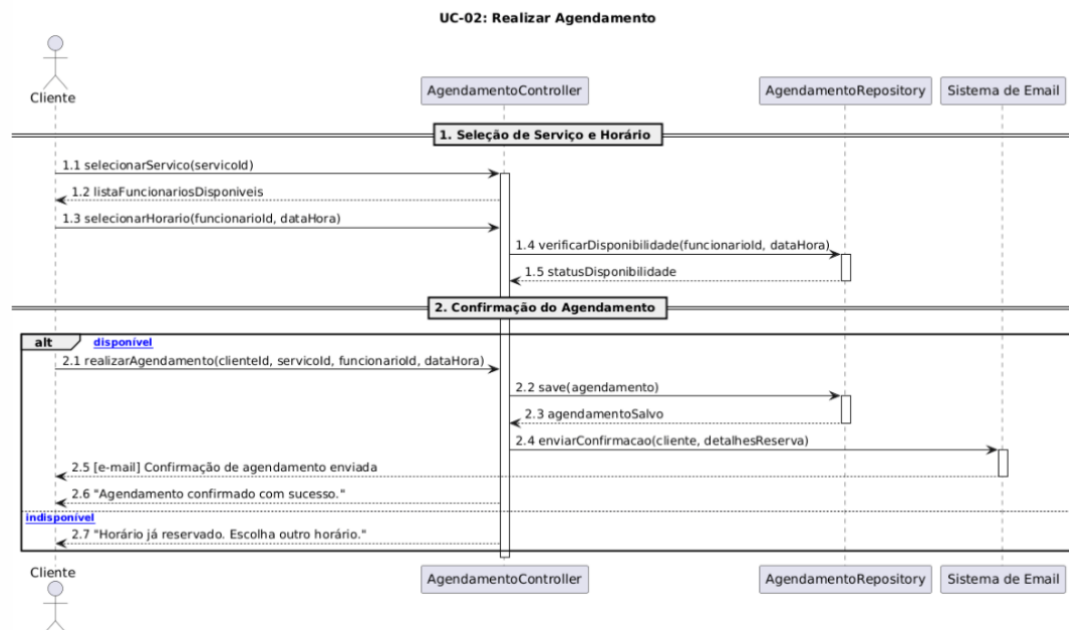


## 2.3 Diagrama de Sequência do Sistema e Contratos de Operação:

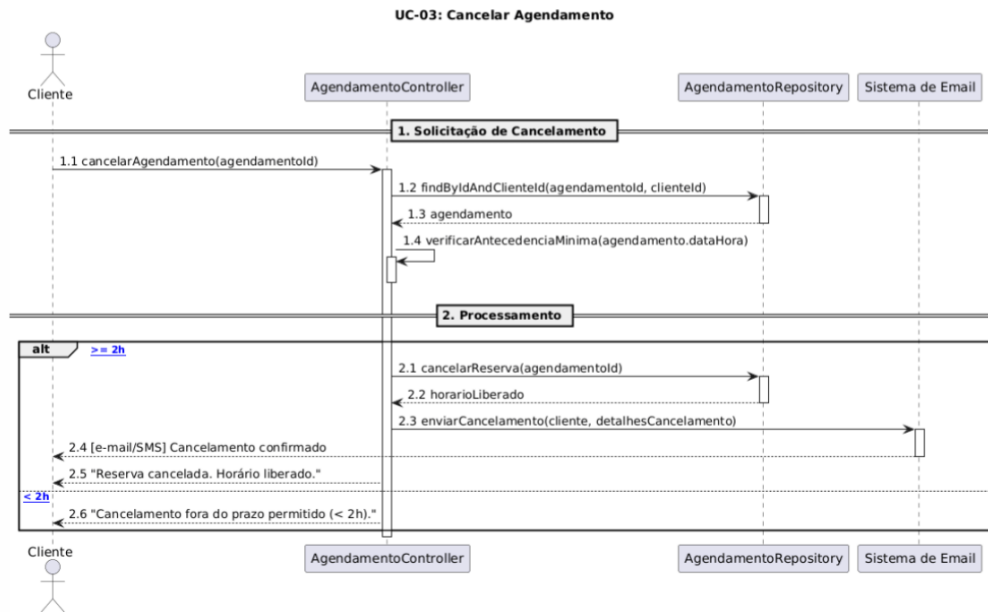


**UC-02: Realizar Agendamento****Contrato de Operação:**

|                             |                                                                                           |
|-----------------------------|-------------------------------------------------------------------------------------------|
| <b>Contrato</b>             | realizarAgendamento                                                                       |
| <b>Operação</b>             | realizarAgendamento(clienteId, servicoId, funcionarioId, dataHora)                        |
| <b>Referências cruzadas</b> | UC-02                                                                                     |
| <b>Pré-condições</b>        | Cliente cadastrado e autenticado; serviço e funcionário disponíveis no horário solicitado |
| <b>Pós-condições</b>        | Agendamento criado com status "pendente"; horário bloqueado na agenda do funcionário      |

**UC-03: Cancelar Agendamento****Contrato de Operação:**

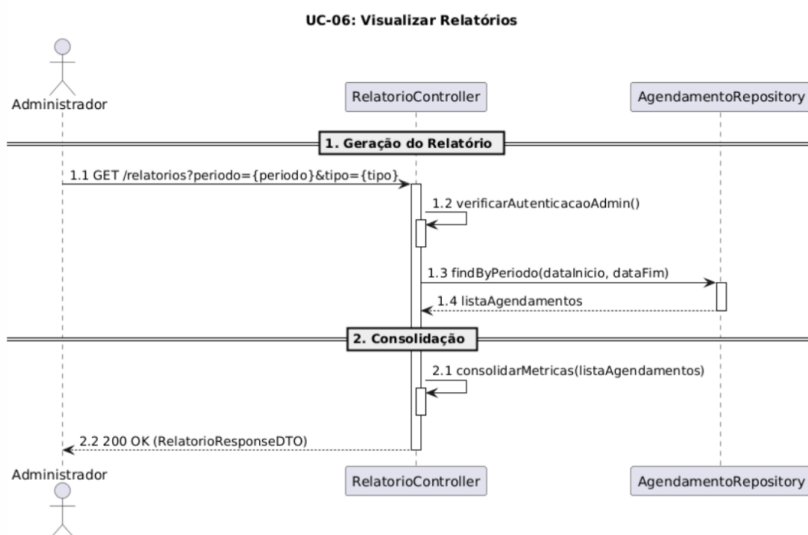
|                             |                                                                                         |
|-----------------------------|-----------------------------------------------------------------------------------------|
| <b>Contrato</b>             | cancelarAgendamento                                                                     |
| <b>Operação</b>             | cancelarAgendamento(agendamentoId, clienteId)                                           |
| <b>Referências cruzadas</b> | UC-03                                                                                   |
| <b>Pré-condições</b>        | Agendamento existe e pertence ao cliente; cancelamento com no mínimo 2h de antecedência |
| <b>Pós-condições</b>        | Agendamento com status "cancelado"; horário liberado na agenda do funcionário           |



## UC-06: Visualizar Relatórios

### Contrato de Operação:

|                             |                                        |
|-----------------------------|----------------------------------------|
| <b>Contrato</b>             | gerarRelatorio                         |
| <b>Operação</b>             | gerarRelatorio(período, tipo)          |
| <b>Referências cruzadas</b> | UC-06                                  |
| <b>Pré-condições</b>        | Usuário autenticado como Administrador |

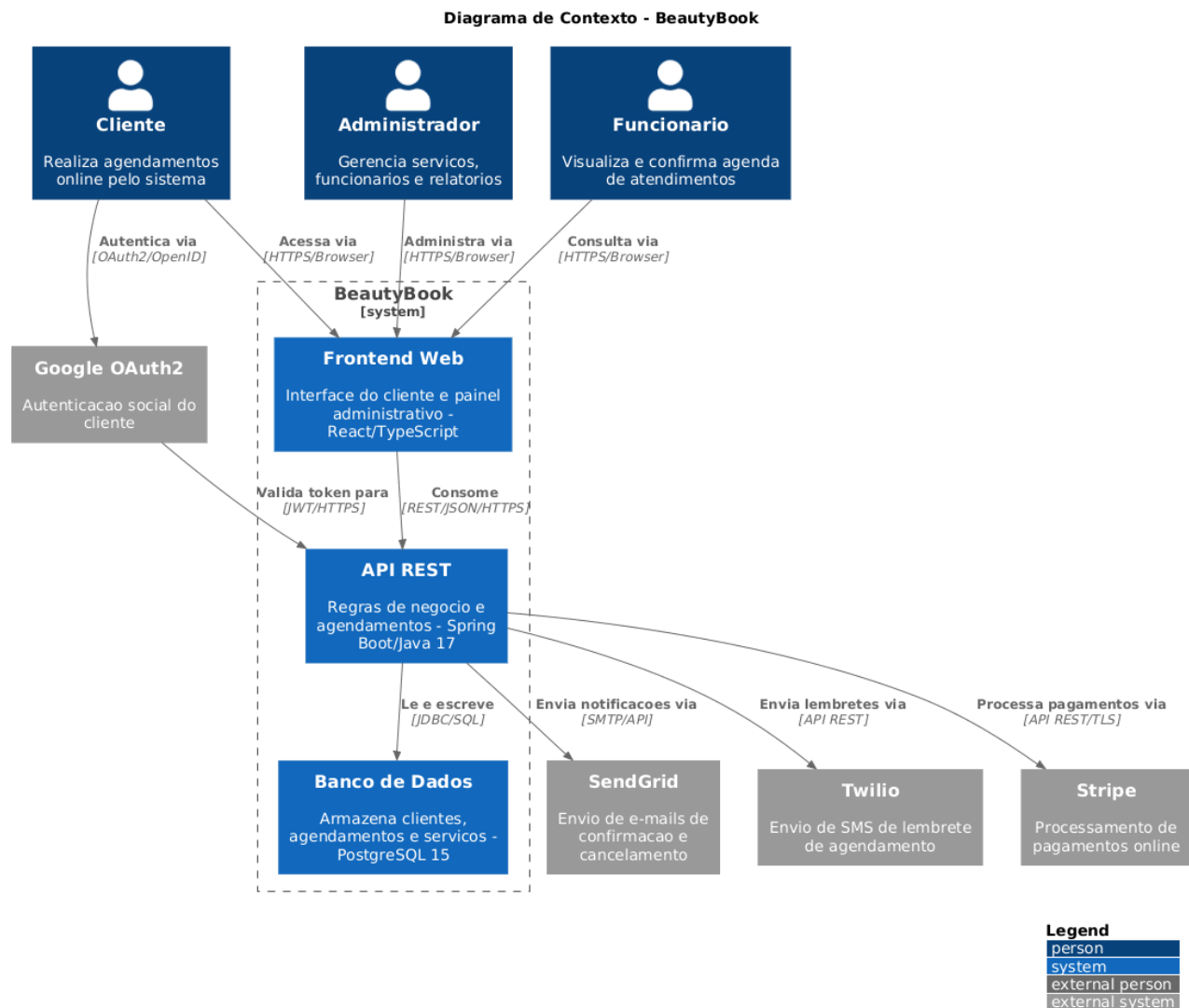




## 3. Modelos de Projeto

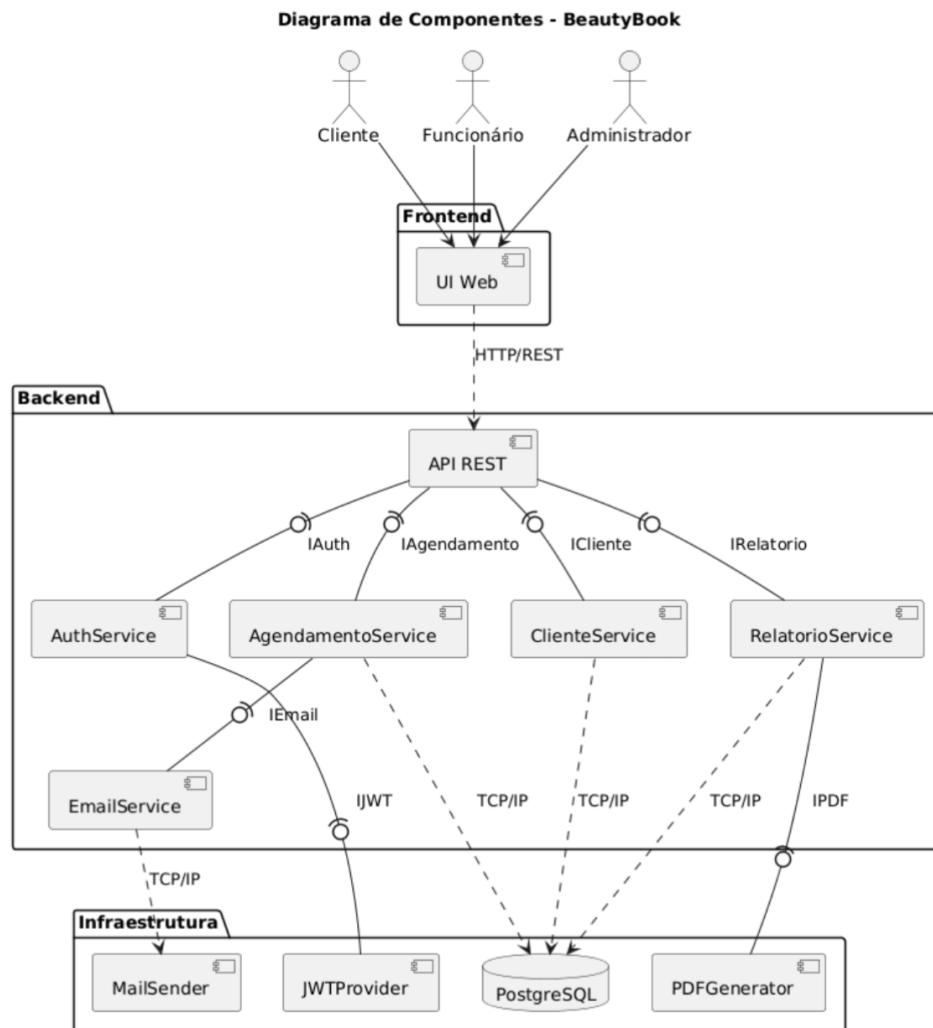
### 3.1 Arquitetura

O sistema BeautyBook segue uma arquitetura em camadas (Layered Architecture), dividida em: Presentation Layer (frontend web em React), Application Layer (API REST em Spring Boot), Domain Layer (regras de negócio) e Infrastructure Layer (banco de dados PostgreSQL, serviço de e-mail). A comunicação entre frontend e backend é feita via HTTP/JSON.

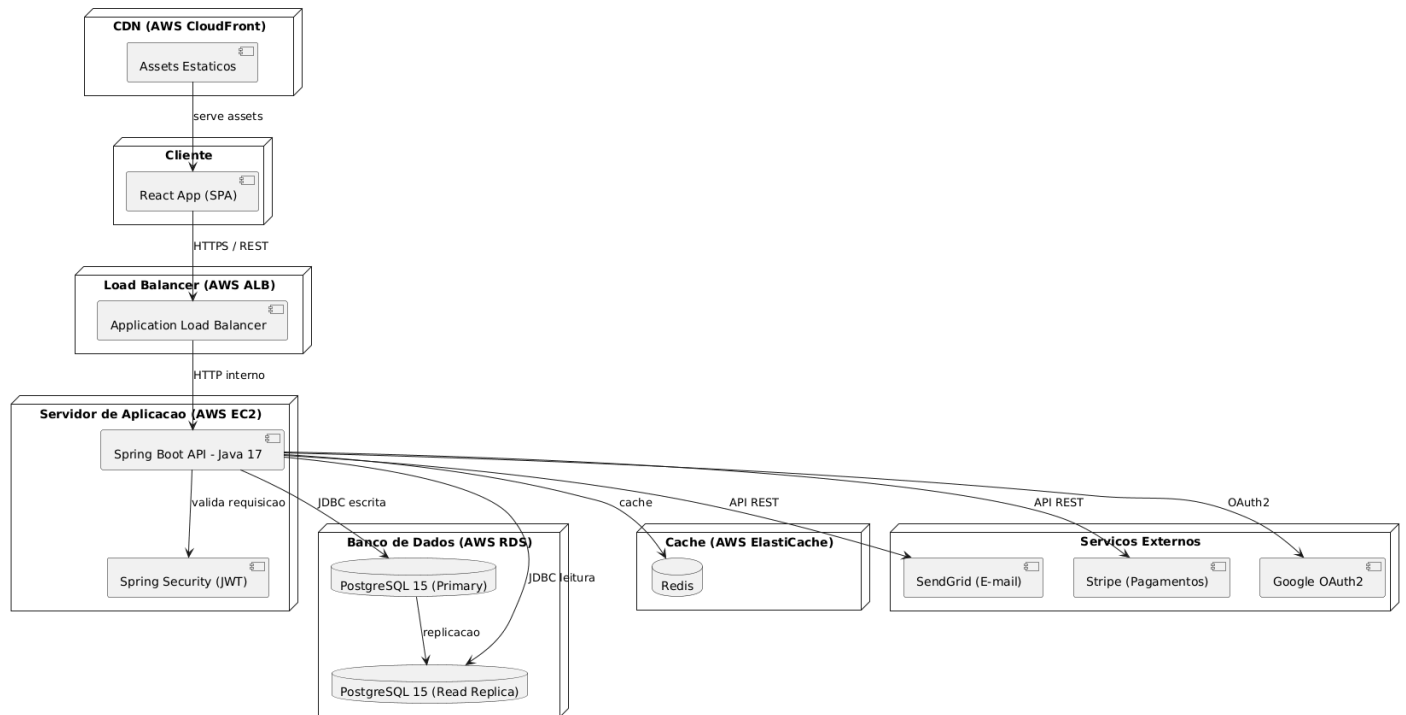


### 3.2 Diagrama de Componentes e Implantação.

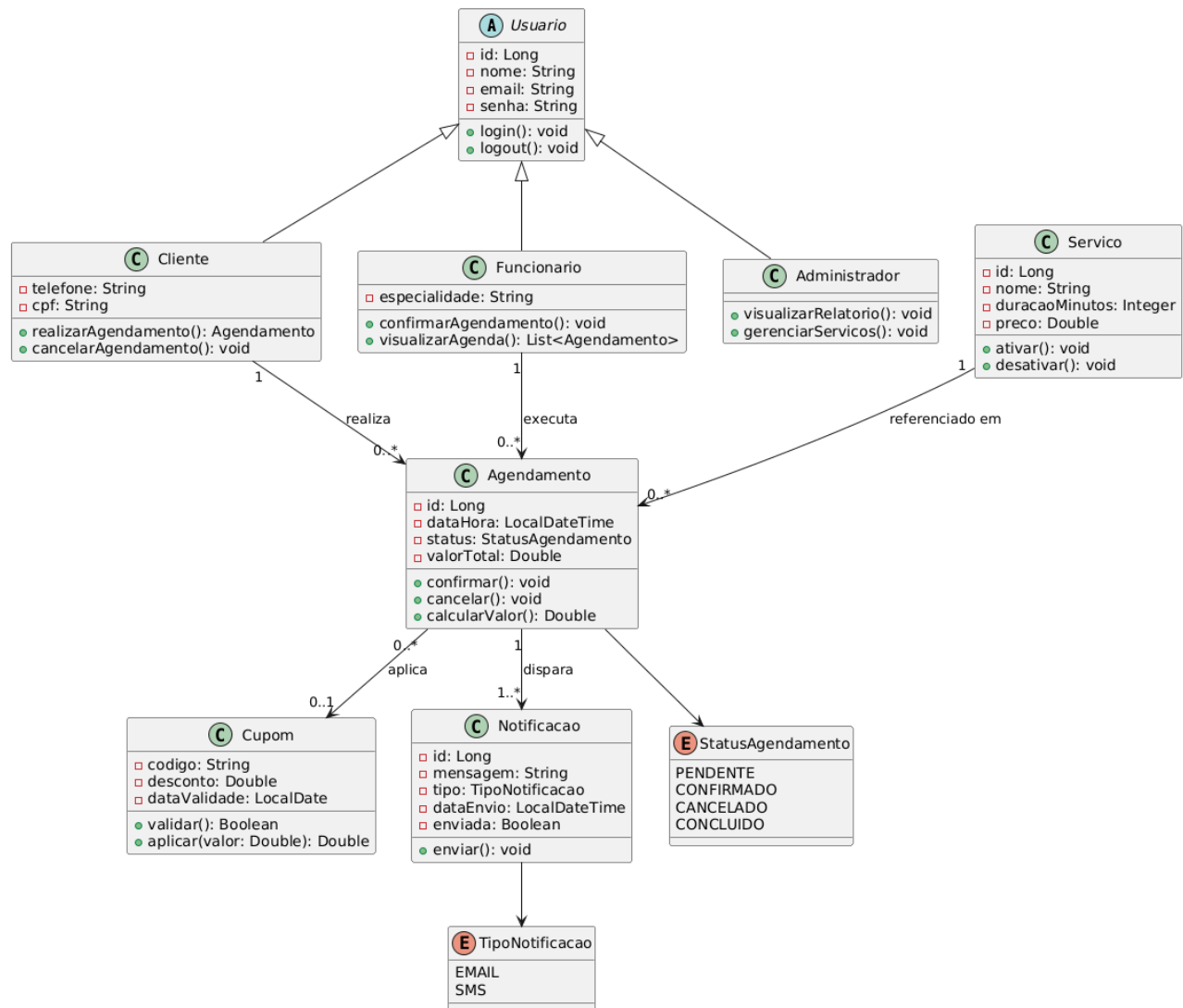
Diagrama de Componentes:



## Diagrama de Implantação:

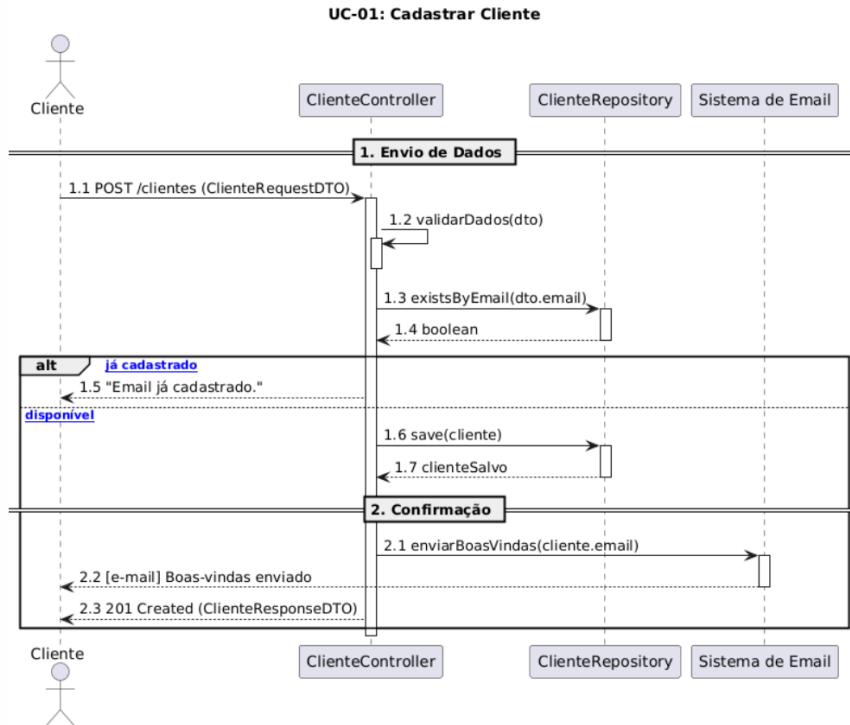


### 3.3 Diagrama de Classes

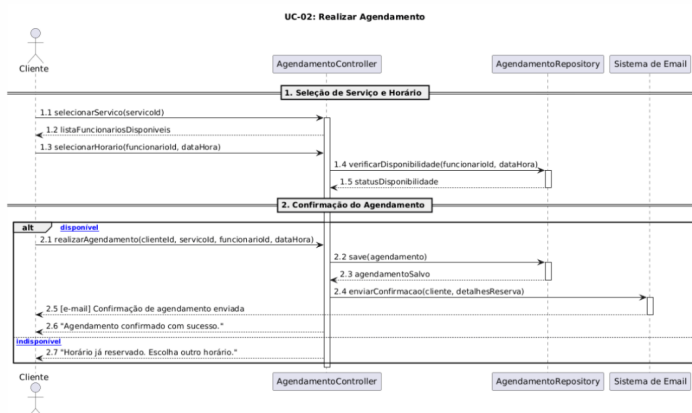


### 3.4 Diagramas de Sequência

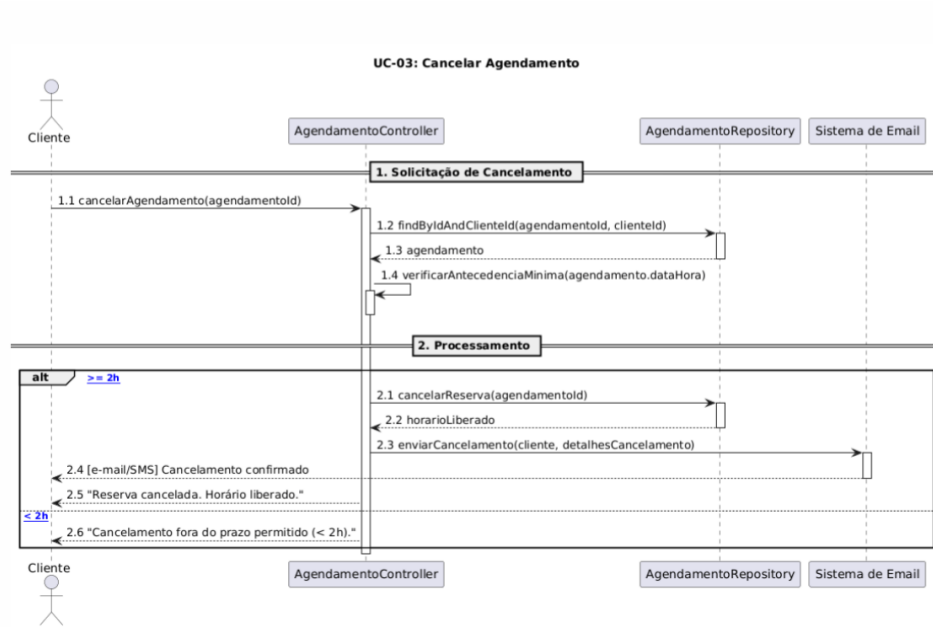
#### UC-01



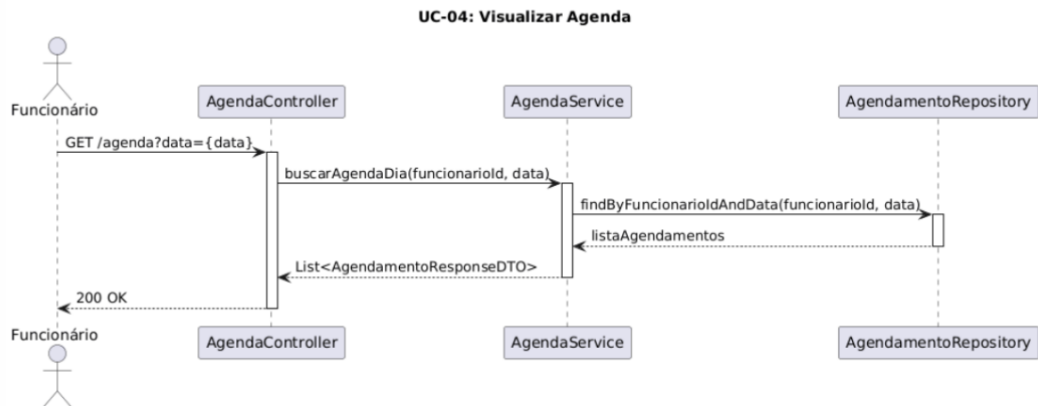
#### UC-02



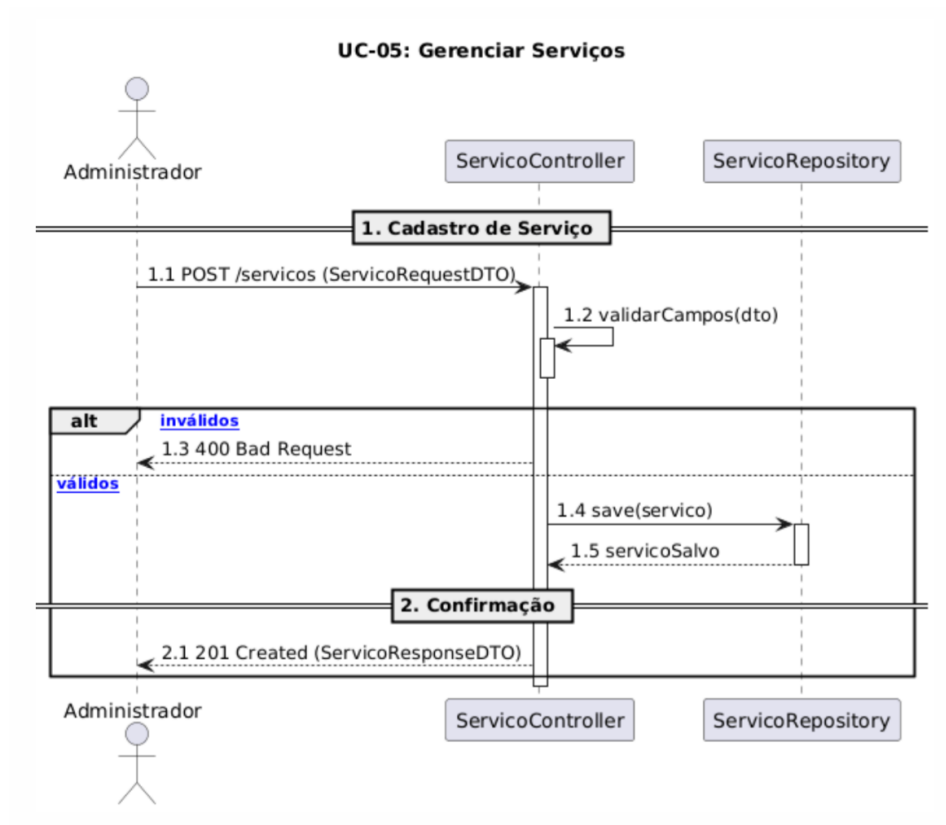
## UC-03



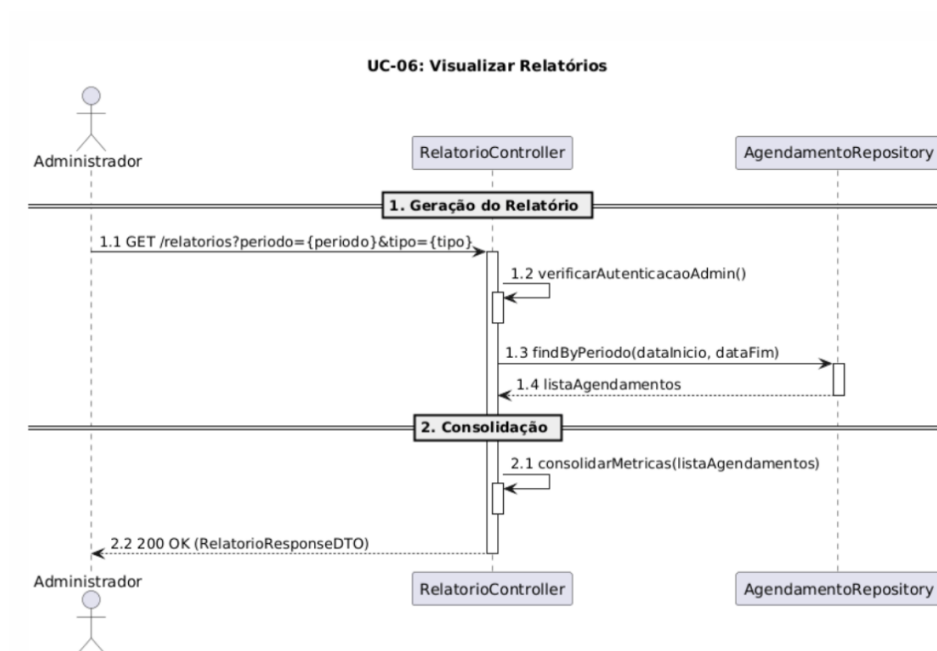
## UC-04



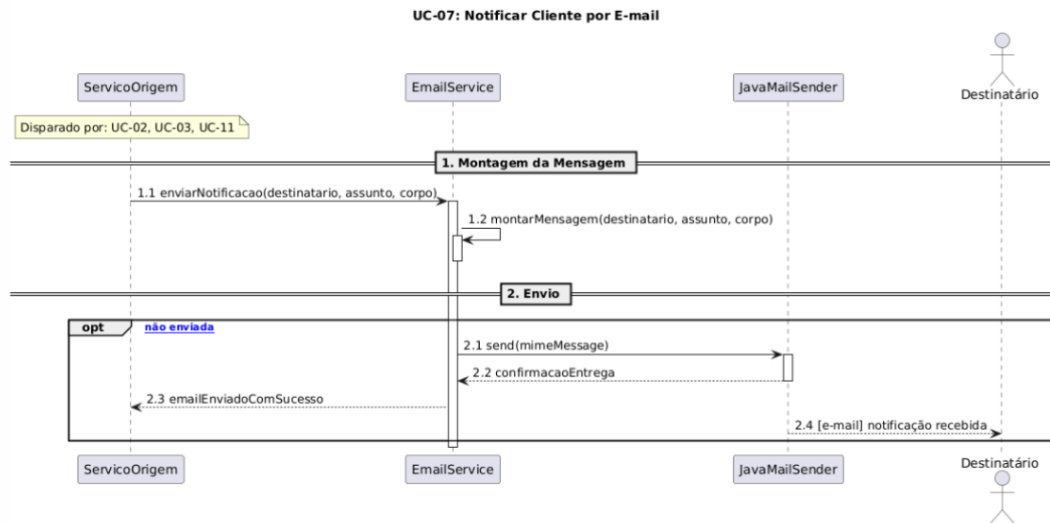
## UC-05



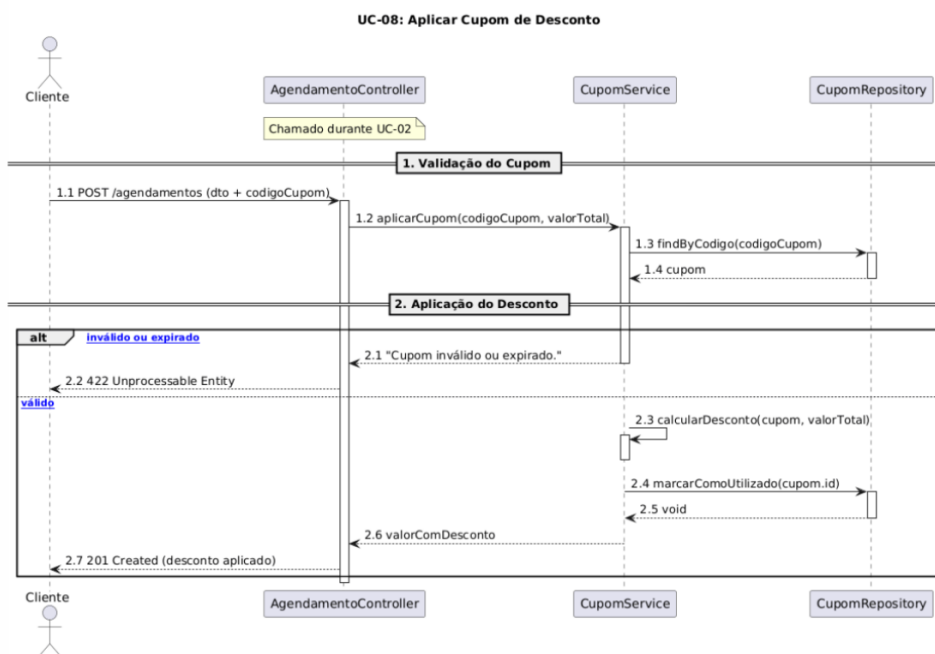
## UC-06



## UC-07

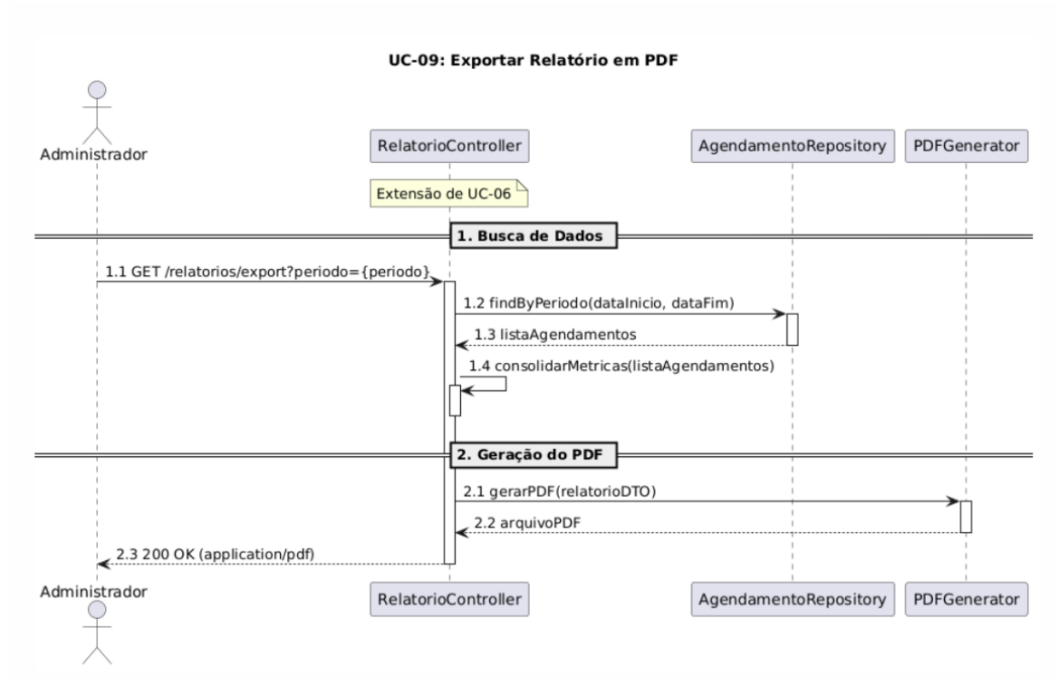


## UC-08

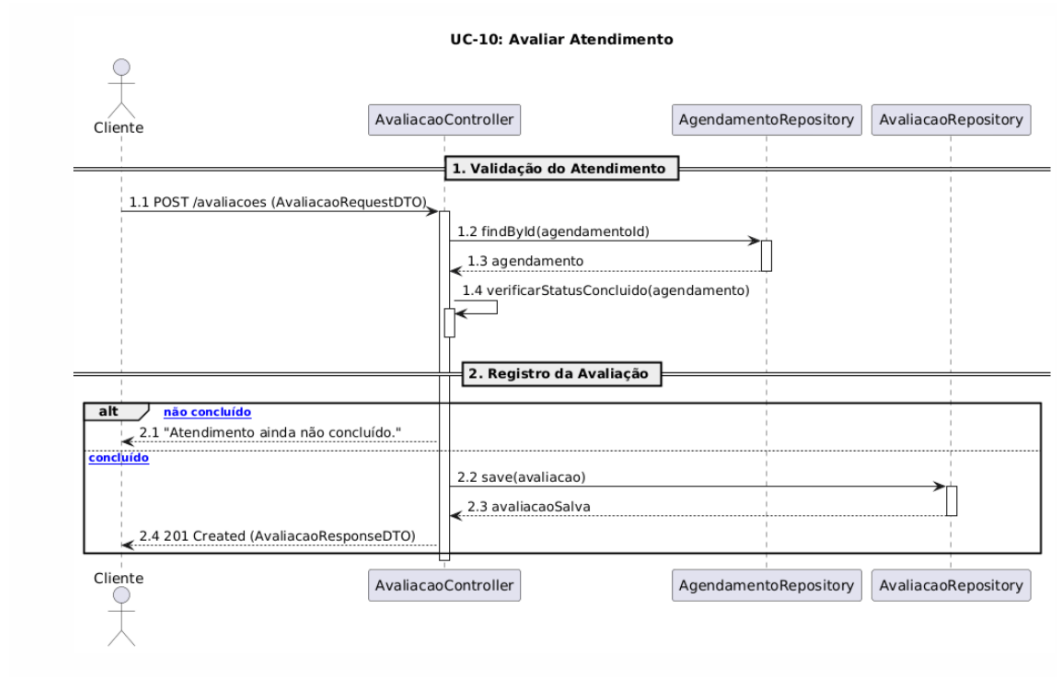




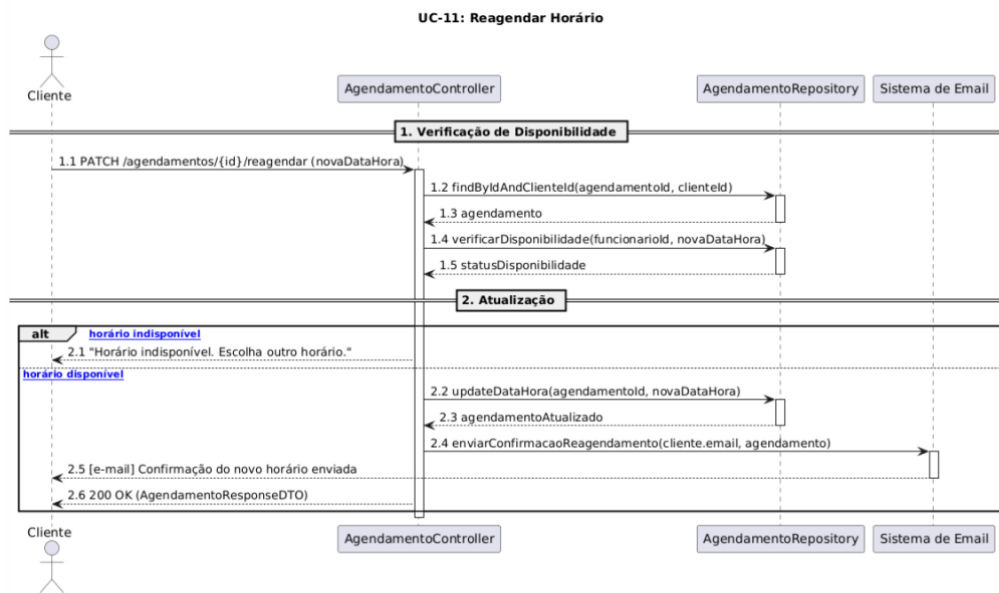
## UC-09



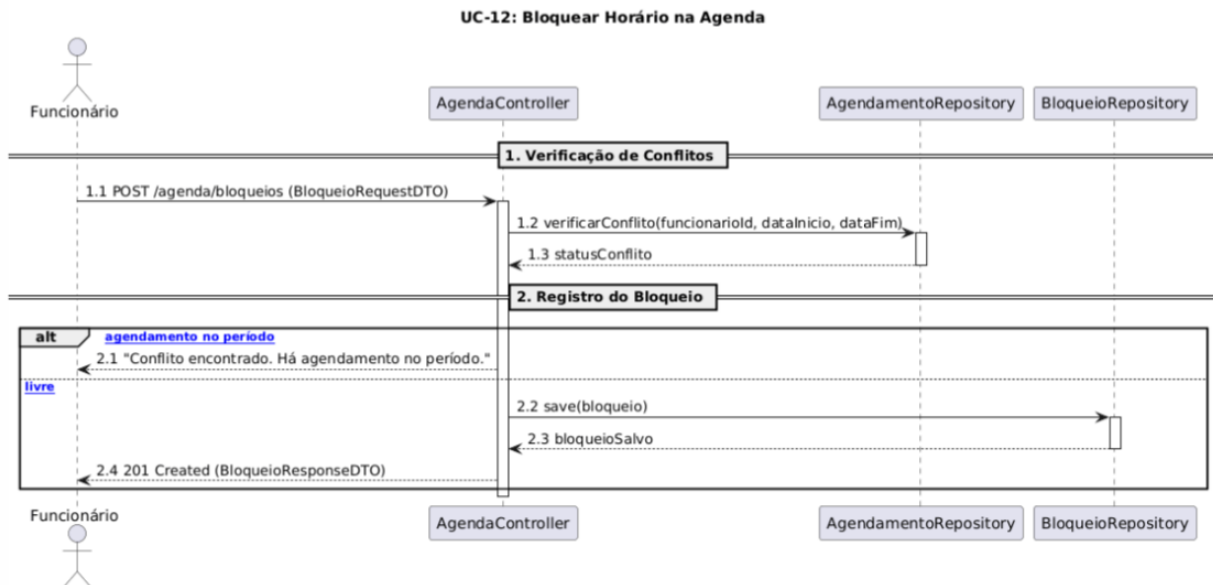
## UC-10



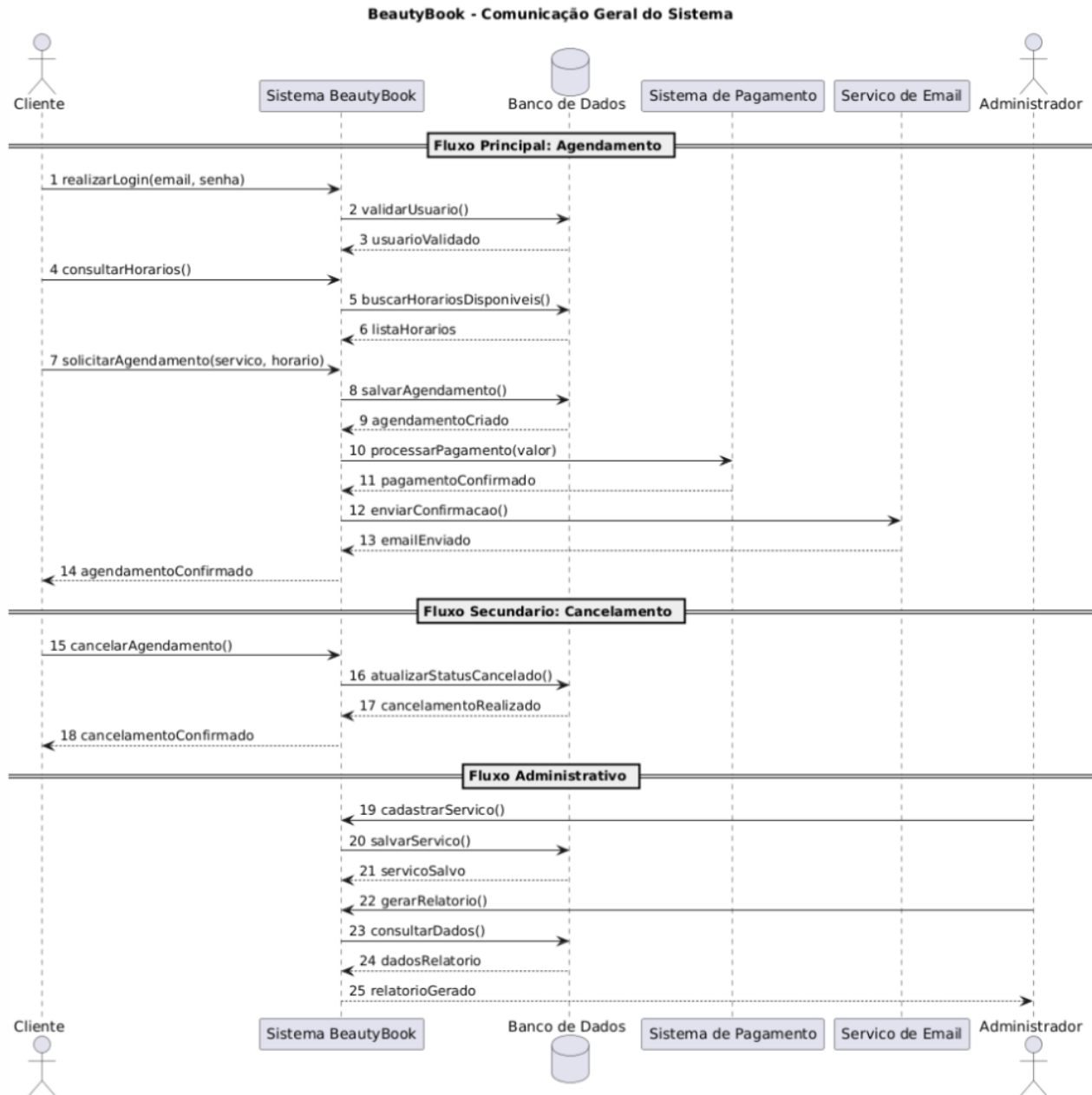
## UC-11



## UC-12



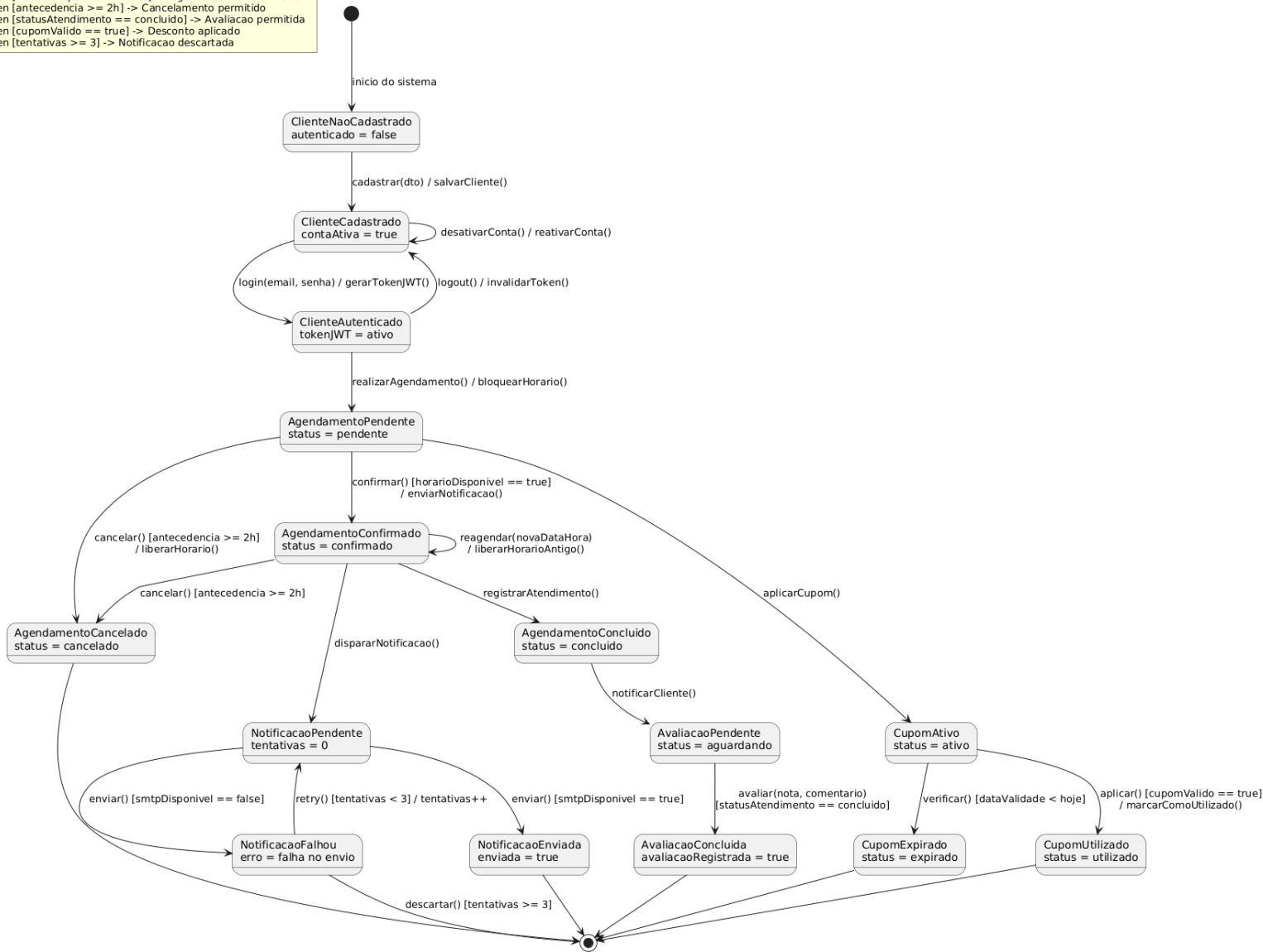
### 3.5 Diagramas de Comunicação



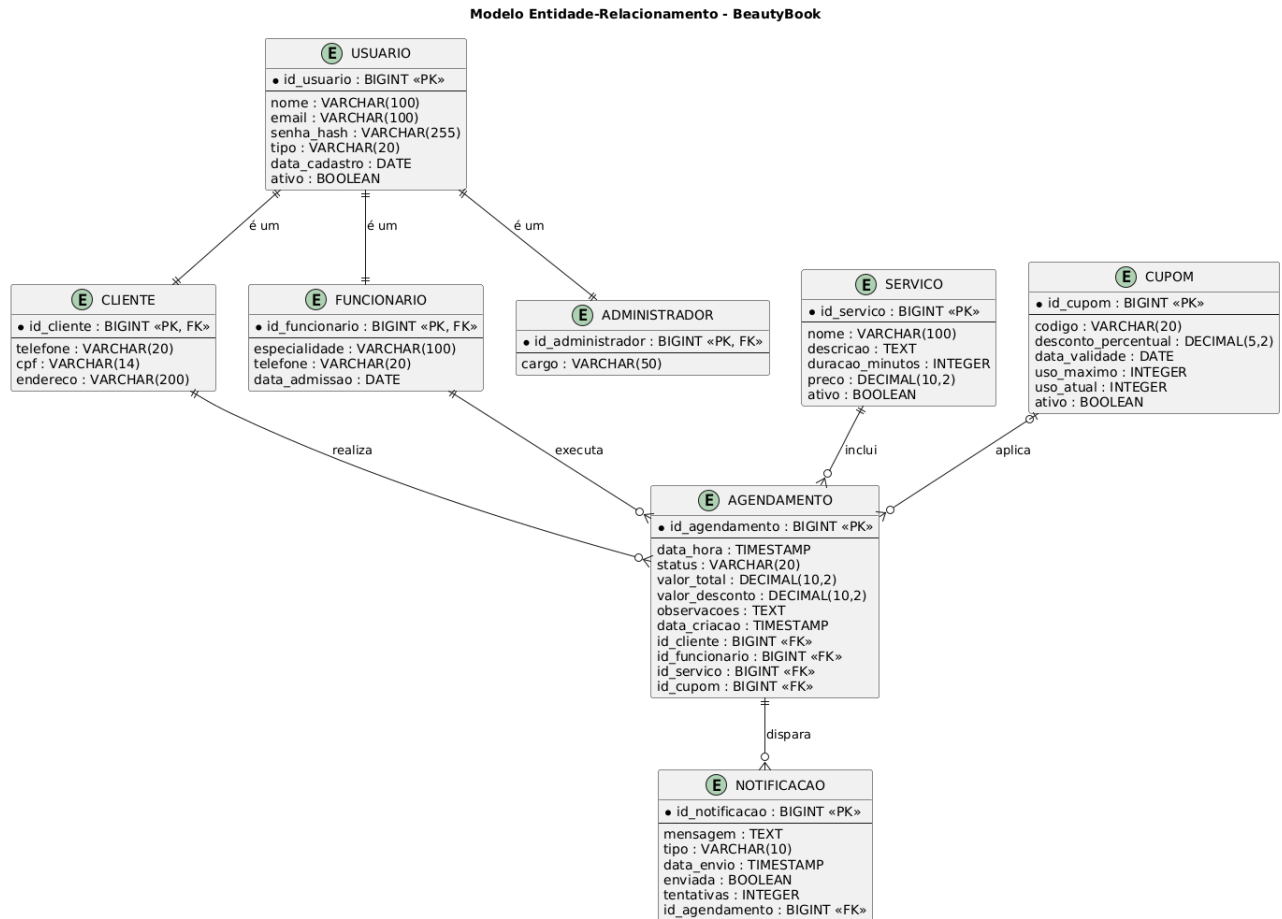
### 3.6 Diagramas de Estados

Diagrama de Transição de Estados  
Sistema BeautyBook

Guards condicionais (when):  
 when [horarioDisponivel == true] -> Agendamento confirmado  
 when [antecedencia >= 2h] -> Cancelamento permitido  
 when [statusAtendimento == concluido] -> Avaliacao permitida  
 when [cupomValido == true] -> Desconto aplicado  
 when [tentativas >= 3] -> Notificacao descartada



## 4. Modelos de Dados



O sistema BeautyBook é estruturado em torno da entidade central AGENDAMENTO, que concentra as principais relações do domínio. O modelo conta com 7 entidades: USUARIO, CLIENTE, FUNCIONARIO, ADMINISTRADOR, SERVICIO, CUPOM e NOTIFICACAO.

### Esquema do Banco de Dados

#### USUARIO — entidade pai por herança

USUARIO (id\_usuario PK, nome, email, senha\_hash, tipo, data\_cadastro, ativo)

#### CLIENTE — especialização de USUARIO

CLIENTE (id\_cliente PK FK, telefone, cpf, endereco)

#### FUNCIONARIO — especialização de USUARIO

FUNCIONARIO (id\_funcionario PK FK, especialidade, telefone, data\_admissao)

#### ADMINISTRADOR — especialização de USUARIO

ADMINISTRADOR (id\_administrador PK FK, cargo)

## SERVICO

SERVICO (id\_servico PK, nome, descricao, duracao\_minutos, preco, ativo)

## CUPOM

CUPOM (id\_cupom PK, codigo, desconto\_percentual, data\_validade, uso\_maximo, uso\_atual, ativo)

## AGENDAMENTO

AGENDAMENTO (id\_agendamento PK, data\_hora, status, valor\_total, valor\_desconto, observacoes, data\_criacao, id\_cliente FK, id\_funcionario FK, id\_servico FK, id\_cupom FK)

## NOTIFICACAO

NOTIFICACAO (id\_notificacao PK, mensagem, tipo, data\_envio, enviada, tentativas, id\_agendamento FK)

## Estratégia de Herança — Mapeamento USUARIO

O modelo adota a estratégia Table Per Subclass (herança por tabela por subclasse), onde:

- A tabela USUARIO armazena os atributos comuns a todos os tipos de usuário.
- Cada subclasse (CLIENTE, FUNCIONARIO, ADMINISTRADOR) possui sua própria tabela com atributos específicos.
- A FK em cada subclasse referencia o id\_usuario da tabela pai.
- O campo tipo em USUARIO discrimina o subtipo ("CLIENTE", "FUNCIONARIO", "ADMINISTRADOR").

Em JPA/Spring Boot, isso é mapeado com a anotação `@Inheritance(strategy = InheritanceType.JOINED)` na classe pai e `@DiscriminatorColumn(name = "tipo")` para identificar o subtipo. Cada subclasse estende a entidade pai com `extends Usuario`.

## Relacionamentos e Cardinalidades

| Relacionamento            | Cardinalidade | Descrição                                        |
|---------------------------|---------------|--------------------------------------------------|
| USUARIO → CLIENTE         | 1:1           | Um usuário pode ser um cliente                   |
| USUARIO → FUNCIONARIO     | 1:1           | Um usuário pode ser um funcionário               |
| USUARIO → ADMINISTRADOR   | 1:1           | Um usuário pode ser um administrador             |
| CLIENTE → AGENDAMENTO     | 1:N           | Um cliente realiza vários agendamentos           |
| FUNCIONARIO → AGENDAMENTO | 1:N           | Um funcionário executa vários agendamentos       |
| SERVICO → AGENDAMENTO     | 1:N           | Um serviço é incluído em vários agendamentos     |
| CUPOM → AGENDAMENTO       | 1:N           | Um cupom pode ser aplicado a vários agendamentos |
| AGENDAMENTO → NOTIFICACAO | 1:N           | Um agendamento dispara várias notificações       |

## Estratégia de Mapeamento Objeto-Relacional (ORM)

O mapeamento entre objetos Java e tabelas relacionais é feito via JPA/Hibernate, seguindo as seguintes convenções:

- Entidades mapeadas com `@Entity` e `@Table(name = "nome_tabela")`
- PKs geradas automaticamente com `@GeneratedValue(strategy = GenerationType.IDENTITY)`
- FKs mapeadas com `@ManyToOne` e `@JoinColumn(name = "id_fk")`
- Relacionamentos bidirecionais usam `@OneToMany(mappedBy = "entidade")`
- O campo status do AGENDAMENTO é mapeado como `@Enumerated(EnumType.STRING)`, armazenando valores como "PENDENTE", "CONFIRMADO", "CANCELADO" e "CONCLUIDO"
- Campos TIMESTAMP são mapeados com `LocalDateTime` e anotados com `@Column`
- Campos BOOLEAN são mapeados diretamente com o tipo primitivo boolean em Java
- O relacionamento entre AGENDAMENTO e NOTIFICACAO utiliza `@OneToMany(cascade = CascadeType.ALL)`, garantindo que ao persistir um agendamento, suas notificações associadas também sejam persistidas automaticamente